

Theme Builder Guide

How to build, package and install a theme for a Codeware site.

Templates, routes, theme.json, settings and the helpers that read them.

What you need before you start

A theme is a folder of Blade templates plus two small files that sit outside it. That is the whole idea, and the rest of this document is detail on how those parts fit together. You need a copy of the site, permission to write to `resources/views/frontend/themes/`, and `routes/web/`. Nothing else — a theme has no build step, no package manager and no migration. It is files.

```
resources/views/frontend/themes/    <- your theme folder goes here
routes/web/<slug>.php              <- your theme's URLs go here
public/themes/<slug>/              <- your theme's CSS/JS go here (optional)
```

Contents

- 1 What a theme is
- 2 The three places a theme's files live
- 3 theme.json — the manifest and the settings
- 4 The helpers — reading your own values
- 5 settings.blade.php — declaring your fields
- 6 Reading values in your templates
- 7 routes/web/<slug>.php — your URLs
- 8 Colours
- 9 Installing and activating
- 10 Rules that will bite you
- 11 Checklist

1 What a theme is

A theme is a folder of Blade templates under `resources/views/frontend/themes/`, named for the site design it produces. The site stores one setting — `site_theme` — holding the slug of the active theme, and every storefront page renders from that theme's folder and nowhere else.

There is no theme table, no theme model and no theme registry. A folder is a theme because it is a folder, which is why installing one is a matter of dropping it in and why deleting one takes its content with it.

Three things follow from that, and they explain most of how the rest behaves:

- **Themes are self-contained.** Every template a page needs has to exist in the active theme. There is no fallback to another theme — a missing template is a 404, not a page in somebody else's design.
- **A theme owns its own settings.** They live in a `theme.json` inside the theme folder, not in the database, so a theme can be zipped and handed to someone else with its content still in it.
- **The slug is the folder name.** It is what `site_theme` stores, what the `theme_{slug}_*` setting keys are built from, and what the route file is called.

The one thing to get right up front

A theme's templates are only reachable through its routes, and the routes are the one part of a theme that does not live in the theme folder. A theme with no `routes/web/<slug>.php` registers no URLs, so selecting it makes every public page — the homepage included — return a 404. The theme picker now flags a theme that ships no route file.

2 The three places a theme's files live

A theme is one folder, but three of its files are not in it. Knowing which is which before you start saves the most confusing half hour you will spend on this.

In the theme folder

`resources/views/frontend/themes/<slug>/` — the templates, and the file that holds the theme's name and content. Optional extras nest in subfolders: `partials/`, `account/`, `errors/`.

```
resources/views/frontend/themes/photography/
|-- theme.json           # manifest + settings (see chapter 3)
|-- home.blade.php       # the one template every theme needs
|-- page.blade.php       # standalone CMS pages
|-- settings.blade.php   # your Theme Settings panel (optional)
|-- errors/
|   |-- 404.blade.php    # your own not-found page (optional)
|-- partials/
|   |-- header.blade.php
|   |-- footer.blade.php
```

`resources/views/frontend/themes/<slug>/settings.blade.php` is the important one to recognise: drop that single file in and a configuration panel for your theme appears in the admin, built from the fields you declare in it. Chapter 5 is about that file.

Outside the theme folder

`routes/web/<slug>.php` — your theme's URLs. It is outside the folder on purpose: routes are shared, registered application-wide, and are the one piece of a theme that is not self-contained. It is also the one piece that cannot travel inside a zip, which chapter 9 comes back to.

`public/themes/<slug>/` — your theme's own CSS and JavaScript, outside the themes directory because they are served as static files. Reference them with `asset()`. The portfolio theme is the built-in example; the

ecommerce theme ships none and is pure Tailwind.

```
{{!-- a template referencing the theme's own stylesheet --}}  
<link rel="stylesheet" href="{{ asset('themes/photography/style.css') }}">  
<script src="{{ asset('themes/photography/script.js') }}" defer></script>
```

The slug

The folder name is the slug, and it may contain letters, numbers, dashes and underscores and nothing else. Keep it lowercase and short — it ends up in `site_theme`, in every one of your setting keys, in your route filename and in a dozen template paths.

3 theme.json — the manifest and the settings

Every value your theme can be configured with lives in one file at the root of your theme folder. There is no second file and no database table. The file holds two kinds of key: five that describe the theme, and every other key, which configures it.

The manifest

These five keys are fixed in name, and they are what the theme picker shows on the card it draws for your theme.

```
{  
  "name": "Photography",  
  "description": "A one-page theme for photographers, with a gallery and booking enquiries.",  
  "version": "1.0.0",  
  "author": "Your Name",  
  "tags": ["portfolio", "gallery"],  
  "theme_photography_hero_title": "Shoots that sell themselves",  
}
```

Every one of them is optional, and each has a fallback, so a theme with no `theme.json` at all is still a usable theme: it appears under its slug, at version 1.0.0, with no author. A `theme.json` that is not valid JSON is treated as if it were absent rather than as an error — a broken file never takes the storefront down.

Your settings

Every other key in the file is a setting, named `theme_{your-slug}_{field}`. A theme called `photography` stores its hero title under `theme_photography_hero_title`. The prefix is redundant inside a file that is already scoped to one theme, and it stays anyway: it is what tells the admin which theme a key belongs to, and it means a `theme.json` copied out of the themes directory on its own is still readable.

You rarely type these keys by hand. The settings panel in chapter 5 declares a field name and the panel builds the key. When you read a value back, the helper resolves the prefix for you — see the next chapter.

The two shapes of value

A scalar setting is a string, and an unset one is an empty string rather than `null` or an absent key. This is deliberate and it is worth internalising, because it is the single thing that most often confuses people reading a theme:

Blank is not missing

A key the owner saved empty is present-and-blank, and comes back as an empty string. A key that is absent means this theme has no such field. They are different states, so `??` is the wrong operator when you want a fallback — use a default argument, or `?:`.

```
{{-- key absent for this theme -> '' is a wrong-looking fallback, use this --}}
@if (theme_setting('hero_title'))
    <h1>{{ theme_setting('hero_title') }}</h1>
@endif

{{-- key present but the owner left it blank -> fall back explicitly --}}
<h1>{{ theme_setting('hero_title', 'Photography') }}</h1>
```

A list setting is a real JSON array of flat objects — projects, testimonials, education entries. Each entry is stored as one array under one key, and it is edited by a repeater in the settings panel rather than by hand.

```
"theme_photography_projects": [
  { "title": "Harbour at dawn", "image": "/storage/theme-uploads/...", "link": "" },
  { "title": "Studio portrait", "image": "", "link": "https://..." }
]
```

An empty list is a perfectly good value, and the useful one. It means "this section has nothing in it", and it lets a template leave the whole section out of the page rather than render a heading over nothing. The portfolio theme hides every empty section this way.

Editing it by hand

The file is meant to be readable: it is pretty-printed, slashes and unicode are left unescaped, and saves are atomic, so a failed write cannot leave half a file behind. A theme with no file gets one created for it — on install, and from the Theme Settings screen if it ever goes missing — seeded with every field the theme declares, each at its empty default. The file is not cached between requests, so an edit you make in an editor is live on the next page load.

4 The helpers — reading your own values

Reading a theme's own settings is a helper call. You do not name the theme, you do not spell out the `theme_{slug}_` prefix, and you do not need to know where the file lives or what happens if it is missing. The same call works in every theme on the site, which is what lets a partial be shared between them.

```
{{-- in any template of any theme --}}
<h1>{{ theme_setting('hero_title') }}</h1>

@foreach (theme_rows('projects') as $project)
    <article>
        <h2>{{ $project['title'] }}</h2>
        @if (! empty($project['image']))
            
        @endif
    </article>
@endforeach
```

Everything there is to call

Call	Gives you
<code>theme_setting('hero_title')</code>	One setting, as a trimmed string.
<code>theme_setting('hero_title', 'Untitled')</code>	The same, with a fallback for blank or missing.
<code>theme_setting('k', '', 'portfolio')</code>	Read a specific theme's file, not the active one.
<code>theme_rows('projects')</code>	A list setting as a list of field => value maps.
<code>theme_rows('tags', 'title')</code>	Treat a bare string in that list as this field.
<code>theme_json()</code>	The whole theme.json, manifest included.
<code>theme_color('accent_color', '#045b30')</code>	A colour, or the fallback if it is blank or not a hex.
<code>theme_name()</code>	The theme's display name, from its manifest.
<code>theme_manifest()</code>	Name, description, version, author, tags — with fallbacks.
<code>theme_setting_key('hero_title')</code>	The full <code>theme_{slug}_hero_title</code> key.
<code>theme_setting_raw('featured', false)</code>	A non-string value, with its JSON type intact.
<code>theme_slug()</code>	The active theme's folder name.

The last argument of every one of them is an optional theme slug. Pass it and you read that theme's file instead of the active theme's, which is what a partial shared between themes needs — a header used by both `photography` and `portfolio` reads the same call twice with two slugs.

```
@foreach (['photography', 'portfolio'] as $slug)
  <h2>{{ theme_setting('hero_title', '', $slug) }}</h2>
@endforeach
```

Choosing between them

Three decisions, and they are worth getting right because each has a different failure mode:

- `theme_setting()` **for text, always.** Colours, URLs, headings, alt text — anything with a string in it. It trims, and it never hands you a `null` to guard.
- `theme_rows()` **for lists.** It returns a plain list of field => value maps, so a template reads `$row['title']` with no casting. It also survives a hand-edited file that went slightly wrong, which `theme_setting_raw()` on a list would not.
- `theme_setting_raw()` **only for a value that is genuinely not a string** — a boolean, a number, a nested object. Rare, and worth noticing when you reach for it.

Blank lists and blank fields

`theme_rows()` returns an empty array for a list with nothing in it, and that is a state worth designing for rather than an error to guard against. It is how a section disappears from a page:

```
@php($projects = theme_rows('projects'))

@if ($projects)
    <section id="projects">
        <h2>Selected work</h2>
        @foreach ($projects as $project) ... @endforeach
    </section>
@endif
```

The optional second argument to `theme_rows()` is the field a bare string stands for. If somebody hand-edits your `theme.json` and pastes `["Fine Art", "Wedding"]` into a list that should hold objects, pass the identity field and those two strings stay readable as two one-field rows instead of silently disappearing.

Underneath

The helpers are a thin skin over two classes you can also call directly: `ThemeSettings` is the file (read, write, merge, delete), and `Themes` is which theme is active and which templates it ships. Reach for those from PHP — a controller, a job, a seeder. In a template, the helper is the shorter path and the one that keeps working when you copy your theme somewhere else.

5 settings.blade.php — declaring your fields

Add one file, `resources/views/frontend/themes/<slug>/settings.blade.php`, and a panel for your theme appears in Admin → Theme Settings. There is nothing to register. The panel is built from what your file declares, and what it saves goes into your theme's `theme.json`.

This is the part with a contract, so read this chapter even if you skim the rest. Three declarations, three bags, and one required attribute on the root element.

The root element

Your file's root element must carry `wire:key` set to the theme slug. It is not decoration. Your file and the other themes' files are swapped into the same slot on the screen, and without a unique key Livewire morphs one into the other in place and keeps the Alpine state it parsed first — so you switch theme and the panel you land on is the previous theme's, editing the wrong fields.

```
<div wire:key="theme-settings-{{ $themeSlug }}" class="space-y-5">
    ...
</div>
```

`$themeSlug` is passed into your file, so the literal above is the whole requirement. Everything else about the file is yours.

Scalar fields

A text, number, URL or colour. Bind a Flux control to `settings.theme_{your-slug}_{field}`.

```
<flux:field>
    <flux:label>Hero title</flux:label>
    <flux:input wire:model="settings.theme_photography_hero_title"
        placeholder="Shoots that sell themselves" />
    <flux:description>Shown as the page's main heading.</flux:description>
</flux:field>
```

The key has to start with your own slug. The panel discovers fields by scanning your file for `theme_` keys, and it decides which theme each belongs to from that prefix — so `theme_portfolio_hero_title` in a theme called

`photography` is not yours, and the panel will not offer it to you. `theme_setting_key('hero_title')` builds the key for you if you need it in a generated context.

List fields (repeaters)

A list of records — projects, services, testimonials, education. Declared with `<x-admin-repeatable-fields>`, which grows the whole add / remove / reorder interface. It reads from `$repeaters`, not `$settings`.

```
<x-admin-repeatable-fields
  :repeaters="$repeaters"
  setting-key="theme_photography_projects"
  label="Projects"
  empty-title="No projects yet"
  empty-hint="The section is hidden until you add at least one."
  add-label="Add project"
  :max="12"
  :fields="[
    ['name' => 'title', 'label' => 'Title'],
    ['name' => 'image', 'label' => 'Image', 'type' => 'media'],
    ['name' => 'description', 'label' => 'Description', 'type' => 'textarea'],
    ['name' => 'link', 'label' => 'Link', 'placeholder' => 'https://...'],
  ]" />
```

Two things in that tag are load-bearing. `:repeaters="$repeaters"` must be passed — a Blade component cannot see a Livewire property two scopes up, and leaving it off renders an empty list over content the owner can plainly see on their site. And the attribute is `setting-key`, not `key`: that attribute is the declaration that makes this a list rather than a text field, so it has to be tellable apart from one at a glance and to the parser. The first field is the row's identity — a row left blank there is dropped on save — so make it the field a person is most likely to fill in.

File fields

A single image or PDF per setting, uploaded straight from disk rather than picked from the Media Library. Declared with `<x-admin-theme-upload>` and read from `$uploads`.

```
<x-admin-theme-upload
  upload-key="theme_photography_hero_image"
  type="image"
  label="Hero image"
  :value="$settings['theme_photography_hero_image'] ?? ''"
  :pending="$uploads['theme_photography_hero_image'] ?? null" />
```

`type` is `image` (JPG, PNG or WEBP, 4MB) or `pdf` (10MB). On save the file is stored under `theme-uploads/`, its URL is written into the setting, and the file it replaced is deleted — so exactly one file is kept per setting, ever. Use a repeater with a `media` field if you need a picture per list entry.

What you have available

Your file is included with `$settings` (scalars, keyed by their full key), `$repeaters` (lists, keyed by their full key), `$uploads` (pending files) and `$themeSlug`. Admin UI components are available too — `<x-admin-section-card>`, `<x-media-picker>`, the `flux:*` set, and the existing `admin-*` components.

What happens on save

The panel writes a flat JSON object into your `theme.json`, keyed exactly as you declared it, merging into whatever is already there. Your manifest keys, and any key you added by hand, are left alone. Repeaters are written as JSON lists. It is a merge rather than a replace, which is why a field belonging to a section you have not built yet survives every save.

6 Reading values in your templates

The other half of the contract. A field you declared is a value you can read back, by the bare field name and nothing else.

```
@php
// inside any template in the theme
$tagline = theme_setting('hero_tagline');
$email   = theme_setting('contact_email', 'hello@example.com');
$projects = theme_rows('projects');
@endphp
```

Because the helper resolves the active theme, a template in your theme needs no mention of your own slug. That is what makes a template portable: copy `home.blade.php` into a new theme, declare the same field names, and it works without editing a line of the template.

Present sections only

The pattern to reach for with a list is to leave the section out entirely when it is empty. A smaller finished page beats a complete one advertising its own gaps — an empty "Testimonials" heading reads as broken, and its absence reads as intentional.

```
@php($projects = theme_rows('projects'))

@if ($projects)
    <section id="work" class="py-24">
        <h2 class="pf-mono text-sm uppercase">Selected work</h2>

        <div class="mt-10 grid gap-8 sm:grid-cols-2">
            @foreach ($projects as $project)
                <article>
                    @if (! empty($project['image']))
                        
                    @endif
                    <h3 class="mt-4 text-lg font-semibold">{{ $project['title'] }}</h3>
                    @if (! empty($project['link']))
                        <a href="{{ $project['link'] }}" class="text-sm underline">View</a>
                    @endif
                </article>
            @endforeach
        </div>
    </section>
@endif
```

Sharing a partial across themes

A partial that reads a setting has to say which theme it is reading, because the theme it is *rendered in* is not the only one it might be rendered in. That is the slug argument:

```
{{-- partials/footer.blade.php, shared by several themes --}}
@php
    $footerTagline = theme_setting('footer_tagline', '', $themeSlug);
    $links = theme_rows('footer_links', 'label', $themeSlug);
@endphp
```

`$themeSlug` is a convention the templates in this codebase already pass into their own partials. Whatever you name the variable, pass the theme's slug rather than relying on the active-theme default — the same partial

included twice for two themes is exactly the case the argument exists for.

7 routes/web/<slug>.php — your URLs

Every theme gets one route file, named after its slug. `routes/web.php` loads every `routes/web/*` .php file at boot, each behind a guard that 404s any request the active theme has no template for. The practical effect is that all installed themes' routes are always registered, but only the active theme's are allowed to answer.

Start from the smallest thing that works:

```
<?php

/*
 * routes/web/photography.php — the storefront routes of the photography theme.
 *
 * Registered behind the 'theme' guard by routes/web.php, so these answer only while
 * this theme is the active one.
 */

use App\Http\Controllers\FrontendController;
use Illuminate\Support\Facades\Route;

Route::get('/', [FrontendController::class, 'home'])->name('home');
```

That is a complete theme. `home` is the route name the controller looks up, and `home.blade.php` is the template it renders. Add a route, add the template, repeat.

Route names are the contract

The controller does not know which theme is active. It asks for a template by route name, and your theme is the one that has to have it. The full set of names the application knows:

Route name	Template it renders	Typical URL
home	home.blade.php	/
page	page.blade.php	/ {slug}
shop	shop.blade.php	/shop
products.show	product.blade.php	/product/{slug}
shop.category	category.blade.php	/category/{slug}
shop.brand	brand.blade.php	/brand/{slug}
shop.tag	tag.blade.php	/tag/{slug}
favorites	favorites.blade.php	/favorites
blog	blog.blade.php	/blog
blog.post	post.blade.php	/blog/{slug}
cart	cart.blade.php	/cart
checkout	checkout.blade.php	/checkout
checkout.confirmation	order-confirmation.blade.php	/checkout/{id}
account.dashboard	account/dashboard.blade.php	/account

Route name	Template it renders	Typical URL
<code>account.orders</code>	<code>account/orders.blade.php</code>	<code>/account/orders</code>
<code>account.orders.show</code>	<code>account/order.blade.php</code>	<code>/account/orders/{id}</code>
<code>account.profile</code>	<code>account/profile.blade.php</code>	<code>/account/profile</code>

There is no fallback

`Themes::viewOrFail()` aborts with a 404 when the active theme has no template for a route. It does not reach for another theme — not `ecommerce`, not `default`, nothing. A page your theme does not implement simply does not exist on your site, which is the correct behaviour: silently rendering it in a design the owner never chose is worse than a 404.

Two consequences worth planning around. If you want a shop, you implement every name from `shop` down to `checkout.confirmation` and you implement them or you do not register the routes — half a shop is worse than none. And if you register a route name without shipping its template, that URL 404s on your site specifically, while every other theme keeps serving it.

Menus

A theme's header and footer menus come from `menu_items` rows grouped `Frontend` (or `Portfolio` for a one-pager of section anchors). Links are filtered at render time by route name, so a menu item pointing at a route your theme does not implement is simply not rendered — which means you can leave shared menu rows alone and let each theme show what it can. There is a `Quick Links` group for footer shortcuts, and a menu entry's icon and label are yours to set in the admin.

Errors

Ship `errors/404.blade.php` in your theme folder to take over the not-found page. Without one, the application's own 404 renders. The `ecommerce` and `portfolio` themes both ship their own, and the `portfolio`'s is a good model for a small theme — it is mostly a header, a short message and a link back to the one page you have.

8 Colours

If your theme uses the site's Tailwind design tokens, you get colours from theme settings without writing any CSS. Declare a colour field in your settings panel, give it the field name from the table below, and the value is emitted as a `:root` custom property on every page — after the compiled stylesheet, so it wins the cascade.

This is theme-agnostic on purpose. It is not switched on for one theme and off for the others: any theme that declares these field names gets them applied, and a theme that declares none emits no block at all.

```
{{-- in a settings.blade.php --}}
<flux:field>
  <flux:label>Accent colour</flux:label>
  <flux:input wire:model="settings.theme_photography_accent_color"
    placeholder="#0f5132" class="font-mono" />
</flux:field>
```

Any value that is not a hex colour is treated as unset, so one bad paste leaves the stylesheet's own default in place instead of emitting a `:root` block that silently stops applying. Values are read with the `theme_color()` helper, which does the same check:

```
<div style="--pf-accent: {{ theme_color('accent_color', '#0f5132') }}">
```

The recognised field names

Field name in your settings	CSS variable it drives
accent_color / primary_color	--color-brand
secondary_color	--color-secondary
header_bg_color	--color-sf-header
header_text_color	--color-sf-header-text
nav_bg_color	--color-sf-nav
nav_text_color	--color-sf-nav-text
footer_bg_color	--color-sf-footer
footer_text_color	--color-sf-footer-text
footer_bottom_color	--color-sf-footer-bottom
button_bg_color	--color-sf-button
button_text_color	--color-sf-button-text
price_color	--color-sf-price
heading_color	--color-sf-heading
text_color	--color-sf-text
page_bg_color	--color-page-bg
sale_color	--color-sale

accent_color, or primary_color

--color-brand is answered by either field. Prefer `accent_color`; `primary_color` is there for themes that predate it. Leave both blank and the stylesheet's own default stands, which is normally what you want.

When you want your own palette instead

A theme that does not want the shared tokens at all should not use these field names. Give it a stylesheet of its own in `public/themes/<slug>/style.css` and drive it from its own settings, exactly as the portfolio theme does with its `--pf-*` custom properties. The two approaches do not conflict — the shared tokens are a convenience, not a requirement, and a theme that ignores them simply gets no block emitted.

9 Installing and activating

A theme is installed by uploading a zip. In development you can equally drop the folder in and it appears on the next request — the screen rescans the themes directory, and there is no cache to clear.

The zip

One folder at the root of the archive, and nothing else beside it. The folder name becomes the slug, so name it the slug you want rather than something you intend to rename later.

```
photography.zip
`-- photography/          <- this folder name is the slug
    |-- theme.json
    |-- home.blade.php
    |-- page.blade.php
    |-- settings.blade.php
    `-- partials/
```

The installer checks the archive before anything touches the themes directory: entries that climb out of the folder are rejected, the file count is capped at 2000 and the unpacked size at 50MB, and an existing theme of the same name is never overwritten. Junk that zip tools add — `__MACOSX`, `.DS_Store` — is ignored.

After the install

- **Your `theme.json` is created if you did not ship one**, seeded with every field your settings panel declares, each at its empty default. Install, pick the theme, fill it in — there is no separate recovery step to discover.
- **A `theme.json` you did ship is left completely alone**, manifest and content both. The installer only fills a gap.
- **Your route file is not in the zip and cannot be**. It lives at `routes/web/<slug>.php`, outside the theme folder. Add it before installing, or the theme will install cleanly, appear in the picker, and 404 the site the moment you select it — which the picker now warns you about.

Activating

Admin → Theme Settings → Site Design. Each installed theme is a card; the radio on it is bound to the same `site_theme` setting the storefront reads, so choosing there is choosing the live site. The card shows your manifest, how many templates you shipped, and your tags.

The page reloads on save, because the active theme is read while the response renders and a Livewire update would not re-run that.

Handover, not just installation

A theme folder carries its content, so zipping one and sending it to somebody else sends the settings too. That is the point of keeping them out of the database — but it is worth remembering before you hand over a theme you have filled in with a client's copy, contact details and real testimonials.

10 Rules that will bite you

None of these are enforced with an error message at the point you break them. They are the failure modes that cost an afternoon.

A missing template is a 404, and only for your theme

There is no cross-theme fallback. If a route your site registers is one you did not implement, it 404s while your theme is active and works again when you switch back. Test by activating the theme and clicking through, not by reading the templates.

Keys must carry your own slug

`theme_photography_hero_title` in a theme called `portfolio` is not a field of yours. The panel will not offer it, will not save it, and a template reading it gets `' '`. A theme whose slug is a prefix of another's is matched

longest-first, so `shop` and `shop_plus` do not fight — but only if you spelled your own keys correctly in the first place.

Blank is not missing

Covered in chapter 3 and repeated here because it is the one that reads as a bug in your own template. A key the owner saved empty comes back as `' '`, not as `null` and not as an error. `$x ?? 'Default'` will hand you `' '`; pass the default as `theme_setting()`'s second argument, or use `?:`.

An empty list should hide the section

`theme_rows()` returning `[]` is a valid, useful state. Guard the whole `<section>` on it rather than rendering a heading and nothing under it.

The ``wire:key`` on your settings file's root

Without it, switching themes on the settings screen leaves that screen driving the previous theme's fields. It looks like the panel went blank, or like your theme has no settings. The symptom is nothing like the cause.

``:repeaters`` is not optional

A `<x-admin-repeatable-fields>` without `:repeaters="$repeaters"` renders empty for every theme. The component knows the difference and says so, but the fix is in your file, not in your data — nothing is lost, it is stored correctly and simply not being read.

The route file is not in the theme folder

Said three times on purpose. A theme that is perfect apart from this one file 404s the entire site, and the missing file is not anywhere you would look inside the theme.

No build step, no cache to clear

There is nothing to compile and nothing to bust. A template or a `theme.json` edited on disk is live on the next request. The one thing to remember is that changes to a `routes/web/*.php` file need `php artisan optimize:clear` — route files are loaded once at boot.

11 Checklist

Before you consider a theme done:

- 1 `theme.json` has a `name`, a `description`, a `version` and an `author`.
 - 2 Every template you route to exists in the theme folder. A missing one is a 404, not a fallback.
 - 3 `routes/web/<slug>.php` exists and is in the repository — it is the one file that cannot travel in a zip.
 - 4 Every settings field reads back with `theme_setting()` or `theme_rows()`, never with a hard-coded value.
 - 5 Every section backed by a list is hidden when the list is empty.
 - 6 No template hard-codes your slug, so it survives being copied into another theme.
 - 7 `settings.blade.php` has `wire:key` on its root element and `:repeaters` on every repeater.
 - 8 Blank values are handled with a default argument, not `??`.
 - 9 The theme has been activated and clicked through — including a 404, on mobile, and with the settings panel saved twice.
-

This document is generated from `scripts/theme_builder_guide.py`. If the behaviour described here stops matching the code, the code is right and the document is stale — re-run the script after fixing the text.